TTS Golf App API Documentation

REST API Reference Guide

TTS Golf Development Team

December 2024

Contents

1 Introduction **2**
1.1 Base URL . . . 2
1.2 Authentication . . . 2
1.3 Response Format . . . 2

2 Authentication API **2**
2.1 Request OTP . . . 2
2.2 Verify OTP . . . 3
2.3 Set Password . . . 3
2.4 Login . . . 3
2.5 Register . . . 4
2.6 Refresh Token . . . 4
2.7 Logout . . . 4
2.8 Logout All Devices . . . 5
2.9 Get Current User . . . 5
2.10 Update Profile . . . 5
2.11 Request Magic Link . . . 6
2.12 Magic Link Redirect . . . 6
2.13 Verify Magic Link . . . 6
2.14 Request SMS Magic Link . . . 6
2.15 Verify SMS Magic Link . . . 6

3 Friends API **7**
3.1 Send Friend Request . . . 7
3.2 Get Pending Requests . . . 7
3.3 Get Sent Requests . . . 7
3.4 Respond to Friend Request . . . 8
3.5 Get Friends . . . 8
3.6 Remove Friend . . . 8
3.7 Discover Users . . . 9
3.8 Get User Profile . . . 9
3.9 Block User . . . 9
3.10 Unblock User . . . 9
3.11 Get Blocked Users . . . 9

4 Roster API **10**
4.1 Get Roster . . . 10
4.2 Add to Roster . . . 10
4.3 Remove from Roster . . . 10
4.4 Get On-Deck Players . . . 11
4.5 Toggle On-Deck Status . . . 11
4.6 Search Users . . . 11
4.7 Reorder Roster . . . 11
4.8 Get Member Profile . . . 12
4.9 Get Member History . . . 12
4.10 Update Member Group . . . 12
4.11 Move Member . . . 12
4.12 Roster Groups . . . 13
4.12.1 Get All Groups . . . 13
4.12.2 Get Single Group . . . 13
4.12.3 Create Group . . . 13
4.12.4 Update Group . . . 13
4.12.5 Update Group Sort . . . 14
4.12.6 Delete Group . . . 14
4.12.7 Get Group Members . . . 14
4.12.8 Get Auto-Invite Members . . . 14
4.13 Waiver Wire . . . 15
4.13.1 Get Waiver Wire . . . 15
4.13.2 Add to Waiver Wire . . . 15
4.13.3 Remove from Waiver Wire . . . 15

5 Tee Times API **15**
5.1 Create Tee Time . . . 15
5.2 Get Tee Times . . . 16
5.3 Get Upcoming Tee Times . . . 16
5.4 Get Calendar View . . . 16
5.5 Get Tee Time History . . . 17
5.6 Get Tee Time Details . . . 17
5.7 Cancel Tee Time . . . 17
5.8 Skip Invite . . . 18
5.9 Send On-Deck Invites . . . 18
5.10 Set FCFS Deadline . . . 18

6 Invites API **18**
6.1 Create Invites . . . 19
6.2 Create Invites from Roster . . . 19
6.3 Get Pending Invites . . . 19
6.4 Get Sent Invites . . . 20
6.5 Respond to Invite . . . 20
6.6 Get Invite Responses . . . 20
6.7 Get User Response History . . . 20
6.8 Cancel Invite . . . 21
6.9 Get Waitlist . . . 21

7 Notifications API **21**
7.1 Get Notifications . . . 21
7.2 Get Unread Count . . . 22
7.3 Mark All as Read . . . 22
7.4 Mark as Read . . . 22
7.5 Mark as Responded . . . 22
7.6 Delete Notification . . . 22

8 Attendance API **23**
8.1 Get Attendance . . . 23
8.2 Check In All . . . 23
8.3 Update Individual Attendance . . . 23
8.4 Cancel Participation . . . 24

9 Social Rate API **24**
9.1 Get Current Score . . . 24
9.2 Get Score History . . . 24
9.3 Get User Score . . . 25

10 Draft (QR Code) API **25**
10.1 Get QR Code . . . 25
10.2 Regenerate QR Code . . . 25
10.3 Scan QR Code . . . 26
10.4 Confirm Draft . . . 26

11 Courses API **26**
11.1 Search Courses . . . 26
11.2 Get Nearby Courses . . . 27
11.3 Get Popular Courses . . . 27
11.4 Get Course Details . . . 27
11.5 Get Favorite Courses . . . 28
11.6 Add Favorite Course . . . 28
11.7 Remove Favorite Course . . . 28

12 Error Codes **28**

13 Rate Limiting **29**

14 Webhooks & Notifications **29**

1 Introduction

This document provides comprehensive documentation for the TTS Golf App REST API. The API enables mobile and web clients to interact with the golf tee time management system.

1.1 Base URL

```
https://api.ttsgolf.com/api
```

1.2 Authentication

Most endpoints require authentication via JWT Bearer tokens. Include the token in the Authorization header:

```
Authorization: Bearer <access_token>
```

1.3 Response Format

All responses are returned in JSON format with the following structure:

```
{
  "success": true,
  "data": { ... },
  "message": "Optional message"
}
```

Error responses:

```
{
  "success": false,
  "error": "Error message",
  "code": "ERROR_CODE"
}
```

2 Authentication API

`/api/auth`

2.1 Request OTP

`POST`

`/api/auth/request-otp`
Request a one-time password for phone number verification.
Access: Public
Request Body:

```
{
  "phoneNumber": "+1234567890"
}
```

Validation:

- `phoneNumber`: E.164 format required (+1234567890)

Response:

```
{
  "success": true,
  "message": "OTP sent successfully"
}
```

2.2 Verify OTP

POST

`/api/auth/verify-otp`
Verify OTP and create a session.
Access: Public (Rate Limited)
Request Body:

```
{
  "phoneNumber": "+1234567890",
  "otp": "123456"
}
```

Validation:

- `phoneNumber`: E.164 format
- `otp`: 6 digits

2.3 Set Password

POST

`/api/auth/set-password`
Set password after OTP verification (signup completion).
Access: Public
Request Body:

```
{
  "phoneNumber": "+1234567890",
  "password": "securePassword123"
}
```

Validation:

- `password`: 8-128 characters

2.4 Login

POST

`/api/auth/login`
Login with phone number or username and password.
Access: Public (Rate Limited)
Request Body:

```
{
  "phoneNumber": "+1234567890",
  "username": "johndoe",
  "password": "securePassword123",
  "rememberMe": true
}
```

Notes:

- Either `phoneNumber` or `username` required
- `rememberMe` extends token expiration

Response:

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
    "user": {
      "id": "uuid",
      "phoneNumber": "+1234567890",
      "firstName": "John",
      "lastName": "Doe"
    }
  }
}
```

2.5 Register

POST

/api/auth/register

Direct user registration (passwordless, no OTP required).

Access: Public

Request Body:

```
{
  "phoneNumber": "+1234567890",
  "firstName": "John",
  "lastName": "Doe",
  "username": "johndoe",
  "email": "john@example.com",
  "handicap": 15
}
```

Validation:

- `phoneNumber`: E.164 format (required)
- `firstName`: 1-50 characters (required)
- `lastName`: 1-50 characters (required)
- `username`: 3-50 characters (optional)
- `email`: Valid email (optional)
- `handicap`: 0-54 (optional)

2.6 Refresh Token

POST

/api/auth/refresh

Refresh access token using refresh token.

Access: Public (Rate Limited)

Request Body:

```
{
  "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

2.7 Logout

POST

/api/auth/logout

Logout current session.
Access: Private (Authenticated)

2.8 Logout All Devices

POST

`/api/auth/logout-all`
Logout from all devices.
Access: Private (Authenticated)

2.9 Get Current User

GET

`/api/auth/me`
Get current authenticated user's profile.
Access: Private (Authenticated)
Response:

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "phoneNumber": "+1234567890",
    "firstName": "John",
    "lastName": "Doe",
    "email": "john@example.com",
    "username": "johndoe",
    "handicap": 15,
    "location": "New York, NY",
    "equipment": {},
    "preferences": {}
  }
}
```

2.10 Update Profile

PUT

`/api/auth/profile`
Update user profile.
Access: Private (Authenticated)
Request Body:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "handicap": 12,
  "location": "Los Angeles, CA",
  "equipment": { "driver": "TaylorMade" },
  "preferences": { "notifications": true }
}
```

2.11 Request Magic Link

POST

`/api/auth/magic-link`
Request magic link for email-based authentication.
Access: Public (Rate Limited)
Request Body:

```
{
  "email": "john@example.com"
}
```

2.12 Magic Link Redirect

GET

`/api/auth/magic-link-redirect`
Universal link handler - redirects to app or web.
Access: Public
Query Parameters:

- `token`: Magic link token

2.13 Verify Magic Link

GET

`/api/auth/verify-magic`
Verify magic link token (API endpoint for app).
Access: Public
Query Parameters:

- `token`: Magic link token

2.14 Request SMS Magic Link

POST

`/api/auth/sms-magic-link`
Request magic link via SMS for signup.
Access: Public (Rate Limited)
Request Body:

```
{
  "phoneNumber": "+1234567890"
}
```

2.15 Verify SMS Magic Link

POST

`/api/auth/verify-sms-magic`
Verify SMS magic link token and complete signup.
Access: Public
Request Body:

```
{
  "token": "abc123..."
}
```

3 Friends API

`/api/friends`
All endpoints require authentication.

3.1 Send Friend Request

POST

`/api/friends/request`
Send a friend request to another user.
Access: Private
Request Body:

```
{
  "userId": "target-user-uuid"
}
```

3.2 Get Pending Requests

GET

`/api/friends/requests/pending`
Get pending friend requests received by the user.
Access: Private
Response:

```
{
  "success": true,
  "data": [
    {
      "id": "request-uuid",
      "fromUser": {
        "id": "user-uuid",
        "firstName": "Jane",
        "lastName": "Smith"
      },
      "createdAt": "2024-12-25T10:00:00Z"
    }
  ]
}
```

3.3 Get Sent Requests

GET

`/api/friends/requests/sent`
Get friend requests sent by the user.
Access: Private

3.4 Respond to Friend Request

PUT

`/api/friends/requests/:id/respond`
Accept or decline a friend request.
Access: Private
URL Parameters:

- `id`: Friend request ID

Request Body:

```
{
  "response": "accepted"
}
```

Valid Responses: `accepted`, `declined`

3.5 Get Friends

GET

`/api/friends`
Get user's friends list.
Access: Private
Response:

```
{
  "success": true,
  "data": [
    {
      "id": "friendship-uuid",
      "friend": {
        "id": "user-uuid",
        "firstName": "Jane",
        "lastName": "Smith",
        "handicap": 18
      },
      "createdAt": "2024-12-20T10:00:00Z"
    }
  ]
}
```

3.6 Remove Friend

DELETE

`/api/friends/:id`
Remove a friend from friends list.
Access: Private
URL Parameters:

- `id`: Friend's user ID

Note: This also removes the roster entry for both users.

3.7 Discover Users

`GET`

`/api/friends/discover`
Discover users to add as friends.
Access: Private
Query Parameters:

- `q`: Search query (optional)
- `limit`: Results limit (optional)
- `offset`: Pagination offset (optional)

3.8 Get User Profile

`GET`

`/api/friends/profile/:id`
Get detailed profile of another user.
Access: Private
URL Parameters:

- `id`: User ID

3.9 Block User

`POST`

`/api/friends/block/:id`
Block a user.
Access: Private
URL Parameters:

- `id`: User ID to block

Note: This also removes the roster entry and friendship for both users.

3.10 Unblock User

`DELETE`

`/api/friends/block/:id`
Unblock a previously blocked user.
Access: Private
URL Parameters:

- `id`: User ID to unblock

3.11 Get Blocked Users

`GET`

`/api/friends/blocked`
Get list of blocked users.
Access: Private

4 Roster API

`/api/roster`
All endpoints require authentication.

4.1 Get Roster

`GET`

`/api/roster`
Get user's roster (prioritized list of golf partners).
Access: Private
Response:

```
{
  "success": true,
  "data": [
    {
      "id": "roster-entry-uuid",
      "contactUserId": "user-uuid",
      "contactName": "John Doe",
      "priority": 1,
      "isOnDeck": false,
      "groupId": "group-uuid",
      "user": {
        "id": "user-uuid",
        "firstName": "John",
        "lastName": "Doe",
        "handicap": 12
      }
    }
  ]
}
```

4.2 Add to Roster

`POST`

`/api/roster`
Add a contact to roster.
Access: Private
Request Body:

```
{
  "contactUserId": "user-uuid",
  "contactPhone": "+1234567890",
  "contactName": "John Doe",
  "contactEmail": "john@example.com"
}
```

Note: Either `contactUserId` or `contactPhone` required.

4.3 Remove from Roster

`DELETE`

`/api/roster/:id`
Remove a contact from roster.
Access: Private
URL Parameters:

- `id`: Roster entry ID

4.4 Get On-Deck Players

GET

`/api/roster/on-deck`
Get on-deck players (backup players for tee times).
Access: Private
Response:

```
{
  "success": true,
  "data": [
    {
      "id": "roster-entry-uuid",
      "contactUserId": "user-uuid",
      "contactName": "Jane Smith",
      "isOnDeck": true,
      "user": { ... }
    }
  ]
}
```

4.5 Toggle On-Deck Status

PUT

`/api/roster/:id/on-deck`
Toggle on-deck status for a roster member.
Access: Private
URL Parameters:

- `id`: Roster entry ID

Request Body:

```
{
  "isOnDeck": true
}
```

4.6 Search Users

GET

`/api/roster/search`
Search users to add to roster.
Access: Private
Query Parameters:

- `q`: Search query (min 2 characters)

4.7 Reorder Roster

PUT

`/api/roster/reorder`
Reorder roster (drag & drop).
Access: Private
Request Body:

```
{
  "orderedIds": ["uuid-1", "uuid-2", "uuid-3"]
}
```

4.8 Get Member Profile

GET

/api/roster/:id/profile
Get detailed member profile.
Access: Private
URL Parameters:

- id: Roster entry ID

4.9 Get Member History

GET

/api/roster/:id/history
Get member's playing history with you.
Access: Private
URL Parameters:

- id: Roster entry ID

4.10 Update Member Group

PUT

/api/roster/:id/group
Update member's group assignment.
Access: Private
URL Parameters:

- id: Roster entry ID

Request Body:

```
{
  "groupId": "group-uuid"
}
```

4.11 Move Member

PUT

/api/roster/:id/move
Move member (drag & drop between sections).
Access: Private
URL Parameters:

- id: Roster entry ID

4.12 Roster Groups

4.12.1 Get All Groups

GET

`/api/roster/groups`
Get all roster groups.
Access: Private
Response:

```
{
  "success": true,
  "data": [
    {
      "id": "group-uuid",
      "name": "Weekend Warriors",
      "description": "Saturday morning group",
      "sortPreference": "priority",
      "memberCount": 8
    }
  ]
}
```

4.12.2 Get Single Group

GET

`/api/roster/groups/:id`
Get a single roster group by ID.
Access: Private
URL Parameters:

- `id`: Group ID

4.12.3 Create Group

POST

`/api/roster/groups`
Create a new roster group.
Access: Private
Request Body:

```
{
  "name": "Weekend Warriors",
  "description": "Saturday morning group",
  "memberIds": ["roster-entry-uuid-1", "roster-entry-uuid-2"]
}
```

4.12.4 Update Group

PUT

`/api/roster/groups/:id`
Update a roster group.
Access: Private
URL Parameters:

- `id`: Group ID

Request Body:

```
{
  "name": "Updated Name",
  "description": "Updated description",
  "memberIds": ["roster-entry-uuid-1"]
}
```

4.12.5 Update Group Sort

`PUT`

`/api/roster/groups/:id/sort`
Update group sort preference.
Access: Private
URL Parameters:

- `id`: Group ID

Request Body:

```
{
  "sortPreference": "priority"
}
```

Valid Sort Options: `priority`, `name`, `handicap`, `recent`

4.12.6 Delete Group

`DELETE`

`/api/roster/groups/:id`
Delete a roster group.
Access: Private
URL Parameters:

- `id`: Group ID

4.12.7 Get Group Members

`GET`

`/api/roster/groups/:id/members`
Get members of a specific group.
Access: Private
URL Parameters:

- `id`: Group ID

4.12.8 Get Auto-Invite Members

`GET`

`/api/roster/groups/:id/auto-invite`
Get auto-invite members for tee time creation.
Access: Private
URL Parameters:

- `id`: Group ID

4.13 Waiver Wire

4.13.1 Get Waiver Wire

GET

`/api/roster/waiver-wire`
Get waiver wire entries for a date.
Access: Private
Query Parameters:

- `date`: Date (YYYY-MM-DD)

4.13.2 Add to Waiver Wire

POST

`/api/roster/waiver-wire`
Add yourself to waiver wire (available to play).
Access: Private
Request Body:

```
{
  "date": "2024-12-25",
  "timePreference": "morning",
  "notes": "Available after 9am"
}
```

4.13.3 Remove from Waiver Wire

DELETE

`/api/roster/waiver-wire/:date`
Remove yourself from waiver wire.
Access: Private
URL Parameters:

- `date`: Date (YYYY-MM-DD)

5 Tee Times API

`/api/tee-times`
All endpoints require authentication.

5.1 Create Tee Time

POST

`/api/tee-times`
Create a new tee time.
Access: Private
Request Body:

```
{
  "courseName": "Pebble Beach Golf Links",
  "courseLocation": "Pebble Beach, CA",
  "teeDate": "2024-12-28",
  "teeTime": "08:30",
```

```
  "maxPlayers": 4,
  "inviteType": "roster",
  "notes": "Bring extra balls"
}
```

Validation:

- `courseName`: Required
- `teeDate`: YYYY-MM-DD format
- `teeTime`: HH:MM format (24-hour)
- `maxPlayers`: 1-8 (default: 4)
- `inviteType`: `roster`, `fcfs`, `open`, `manual`

Response:

```
{
  "success": true,
  "data": {
    "id": "tee-time-uuid",
    "courseName": "Pebble Beach Golf Links",
    "teeDate": "2024-12-28",
    "teeTime": "08:30:00",
    "maxPlayers": 4,
    "status": "pending",
    "organizerId": "user-uuid"
  }
}
```

5.2 Get Tee Times

`GET`

`/api/tee-times`
Get user's tee times.
Access: Private
Query Parameters:

- `status`: Filter by status (optional)
- `limit`: Results limit (optional)
- `offset`: Pagination offset (optional)

5.3 Get Upcoming Tee Times

`GET`

`/api/tee-times/upcoming`
Get upcoming tee times.
Access: Private

5.4 Get Calendar View

`GET`

`/api/tee-times/calendar`
Get tee times for calendar view.
Access: Private
Query Parameters:

- `start`: Start date (YYYY-MM-DD) - required
- `end`: End date (YYYY-MM-DD) - required

5.5 Get Tee Time History

GET

`/api/tee-times/history`
Get complete tee time history with attendance status.
Access: Private
Query Parameters:

- `filter`: `all`, `upcoming`, `completed`, `cancelled`, `no_show`
- `limit`: Results limit (optional)
- `offset`: Pagination offset (optional)

5.6 Get Tee Time Details

GET

`/api/tee-times/:id`
Get detailed tee time information.
Access: Private
URL Parameters:

- `id`: Tee time ID

Response:

```
{
  "success": true,
  "data": {
    "id": "tee-time-uuid",
    "courseName": "Pebble Beach Golf Links",
    "courseLocation": "Pebble Beach, CA",
    "teeDate": "2024-12-28",
    "teeTime": "08:30:00",
    "maxPlayers": 4,
    "status": "confirmed",
    "inviteType": "roster",
    "organizer": {
      "id": "user-uuid",
      "firstName": "John",
      "lastName": "Doe"
    },
    "participants": [
      {
        "id": "participant-uuid",
        "user": { ... },
        "status": "accepted"
      }
    ],
    "invites": [ ... ]
  }
}
```

5.7 Cancel Tee Time

POST

`/api/tee-times/:id/cancel`
Cancel a tee time.
Access: Private (Organizer only)
URL Parameters:

- `id`: Tee time ID

5.8 Skip Invite

`POST`

`/api/tee-times/:id/invites/:inviteId/skip`
Skip an invite and advance to next roster member.
Access: Private (Organizer only)
URL Parameters:

- `id`: Tee time ID
- `inviteId`: Invite ID to skip

5.9 Send On-Deck Invites

`POST`

`/api/tee-times/:id/on-deck-invites`
Send FCFS invites to selected On-Deck players.
Access: Private (Organizer only)
URL Parameters:

- `id`: Tee time ID

Request Body:

```
{
  "playerIds": ["user-uuid-1", "user-uuid-2"],
  "deadline": "2024-12-27T18:00:00Z"
}
```

5.10 Set FCFS Deadline

`POST`

`/api/tee-times/:id/fcfs-deadline`
Set FCFS response deadline for all pending invites.
Access: Private (Organizer only)
URL Parameters:

- `id`: Tee time ID

Request Body:

```
{
  "deadline": "2024-12-27T18:00:00Z"
}
```

6 Invites API

`/api/invites`
All endpoints require authentication.

6.1 Create Invites

POST

`/api/invites`
Create invites for a tee time.
Access: Private
Request Body:

```
{
  "teeTimeId": "tee-time-uuid",
  "invitees": [
    { "userId": "user-uuid" },
    { "phone": "+1234567890", "name": "Guest Player" }
  ]
}
```

Validation:

- `teeTimeId`: Valid UUID required
- `invitees`: At least one invitee required
- Each invitee needs either `userId` or `phone`

6.2 Create Invites from Roster

POST

`/api/invites/from-roster`
Create invites from roster (priority-based).
Access: Private
Request Body:

```
{
  "teeTimeId": "tee-time-uuid",
  "rosterIds": ["roster-entry-uuid-1", "roster-entry-uuid-2"],
  "count": 3
}
```

Notes:

- If `rosterIds` provided, invites those specific roster members
- If `count` provided without `rosterIds`, invites top N from roster

6.3 Get Pending Invites

GET

`/api/invites/pending`
Get pending invites received by the user.
Access: Private
Response:

```
{
  "success": true,
  "data": [
    {
      "id": "invite-uuid",
      "teeTime": {
        "id": "tee-time-uuid",
        "courseName": "Pebble Beach",
        "teeDate": "2024-12-28",
```

```
        "teeTime": "08:30:00"
      },
      "organizer": {
        "id": "user-uuid",
        "firstName": "John"
      },
      "status": "pending",
      "expiresAt": "2024-12-27T18:00:00Z",
      "createdAt": "2024-12-25T10:00:00Z"
    }
  ]
}
```

6.4 Get Sent Invites

GET

`/api/invites/sent`
Get invites sent by the user.
Access: Private

6.5 Respond to Invite

PUT

`/api/invites/:id/respond`
Respond to an invite (accept/decline/maybe).
Access: Private
URL Parameters:

- `id`: Invite ID

Request Body:

```
{
  "response": "accepted",
  "message": "Looking forward to it!"
}
```

Valid Responses: `accepted`, `declined`, `maybe`

6.6 Get Invite Responses

GET

`/api/invites/:id/responses`
Get response history for an invite.
Access: Private
URL Parameters:

- `id`: Invite ID

6.7 Get User Response History

GET

`/api/invites/history`
Get user's invite response history.
Access: Private

6.8 Cancel Invite

DELETE

`/api/invites/:id`
Cancel an invite.
Access: Private (Organizer only)
URL Parameters:

- `id`: Invite ID

6.9 Get Waitlist

GET

`/api/invites/:id/waitlist`
Get waitlist for a tee time.
Access: Private
URL Parameters:

- `id`: Invite ID

7 Notifications API

`/api/notifications`
All endpoints require authentication.

7.1 Get Notifications

GET

`/api/notifications`
Get user's notifications.
Access: Private
Query Parameters:

- `limit`: Results limit (optional)
- `offset`: Pagination offset (optional)
- `unreadOnly`: Filter to unread only (optional)

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "notification-uuid",
      "type": "friend_request",
      "title": "New Friend Request",
      "body": "John Doe wants to be your friend",
      "data": { "requestId": "uuid" },
      "isRead": false,
      "isResponded": false,
      "createdAt": "2024-12-25T10:00:00Z"
    }
  ]
}
```

7.2 Get Unread Count

GET

`/api/notifications/unread-count`
Get count of unread notifications.
Access: Private
Response:

```
{
  "success": true,
  "data": {
    "count": 5
  }
}
```

7.3 Mark All as Read

PUT

`/api/notifications/read-all`
Mark all notifications as read.
Access: Private

7.4 Mark as Read

PUT

`/api/notifications/:id/read`
Mark a specific notification as read.
Access: Private
URL Parameters:

- `id`: Notification ID

7.5 Mark as Responded

PUT

`/api/notifications/:id/responded`
Mark a notification as responded (for actionable notifications like friend requests).
Access: Private
URL Parameters:

- `id`: Notification ID

Note: Used to persist response state for notifications that require user action.

7.6 Delete Notification

DELETE

`/api/notifications/:id`
Delete a notification.
Access: Private
URL Parameters:

- `id`: Notification ID

8 Attendance API

`/api/tee-times/:id/...`
All endpoints require authentication.

8.1 Get Attendance

`GET`

`/api/tee-times/:id/attendance`
Get attendance status for all participants.
Access: Private (Organizer only)
URL Parameters:

- `id`: Tee time ID

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "attendance-uuid",
      "userId": "user-uuid",
      "user": {
        "firstName": "John",
        "lastName": "Doe"
      },
      "status": "pending",
      "checkedInAt": null
    }
  ]
}
```

8.2 Check In All

`POST`

`/api/tee-times/:id/check-in/all`
One-tap check-in: Mark all participants as present.
Access: Private (Organizer only)
URL Parameters:

- `id`: Tee time ID

8.3 Update Individual Attendance

`POST`

`/api/tee-times/:id/check-in/:attendanceId`
Mark individual participant status.
Access: Private (Organizer only)
URL Parameters:

- `id`: Tee time ID
- `attendanceId`: Attendance record ID

Request Body:

```
{
  "status": "present"
}
```

Valid Statuses: `present`, `no_show`

8.4 Cancel Participation

`POST`

`/api/tee-times/:id/cancel-participation`
User cancels their own participation.
Access: Private
URL Parameters:

- `id`: Tee time ID

9 Social Rate API

`/api/social-rate`
All endpoints require authentication. Social Rate is a reputation score based on attendance and reliability.

9.1 Get Current Score

`GET`

`/api/social-rate`
Get current user's social rate score and summary.
Access: Private
Response:

```
{
  "success": true,
  "data": {
    "score": 85,
    "level": "Reliable",
    "totalRounds": 25,
    "attendedRounds": 23,
    "noShows": 2,
    "attendanceRate": 92
  }
}
```

9.2 Get Score History

`GET`

`/api/social-rate/history`
Get user's score change history.
Access: Private
Query Parameters:

- `limit`: Results limit (default: 50)
- `offset`: Pagination offset (default: 0)

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "history-uuid",
      "previousScore": 80,
      "newScore": 85,
      "change": 5,
      "reason": "Attended tee time",
      "teeTimeId": "tee-time-uuid",
      "createdAt": "2024-12-25T10:00:00Z"
    }
  ]
}
```

9.3 Get User Score

GET

`/api/social-rate/user/:userId`
Get another user's public social rate score.
Access: Private
URL Parameters:

- `userId`: User ID

10 Draft (QR Code) API

`/api/draft`
All endpoints require authentication. The Draft feature allows users to add new contacts via QR code scanning.

10.1 Get QR Code

GET

`/api/draft/qr`
Get or generate user's QR code for "Get Drafted".
Access: Private
Response:

```
{
  "success": true,
  "data": {
    "qrCode": "base64-encoded-qr-image",
    "qrData": "encrypted-user-data",
    "expiresAt": "2024-12-26T10:00:00Z"
  }
}
```

10.2 Regenerate QR Code

POST

`/api/draft/qr/regenerate`
Regenerate QR code (invalidates previous codes).
Access: Private

10.3 Scan QR Code

POST

/api/draft/scan
Scan and validate a QR code.
Access: Private
Request Body:

```
{
  "qrData": "scanned-qr-data-string"
}
```

Response:

```
{
  "success": true,
  "data": {
    "userId": "scanned-user-uuid",
    "firstName": "Jane",
    "lastName": "Smith",
    "handicap": 18,
    "isAlreadyContact": false
  }
}
```

10.4 Confirm Draft

POST

/api/draft/confirm
Confirm adding scanned user to On-Deck.
Access: Private
Request Body:

```
{
  "userId": "scanned-user-uuid"
}
```

11 Courses API

/api/courses

11.1 Search Courses

GET

/api/courses/search
Search for golf courses.
Access: Public (Optional Auth for personalization)
Query Parameters:

- q: Search query
- limit: Results limit (optional)
- offset: Pagination offset (optional)

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "course-uuid",
      "name": "Pebble Beach Golf Links",
      "location": "Pebble Beach, CA",
      "rating": 4.8,
      "holes": 18,
      "par": 72
    }
  ]
}
```

11.2 Get Nearby Courses

GET

`/api/courses/nearby`
Get golf courses near a location.
Access: Public (Optional Auth for personalization)
Query Parameters:

- `lat`: Latitude
- `lng`: Longitude
- `radius`: Search radius in miles (optional)
- `limit`: Results limit (optional)

11.3 Get Popular Courses

GET

`/api/courses/popular`
Get popular golf courses.
Access: Public (Optional Auth for personalization)
Query Parameters:

- `limit`: Results limit (optional)

11.4 Get Course Details

GET

`/api/courses/:id`
Get detailed course information.
Access: Public (Optional Auth for personalization)
URL Parameters:

- `id`: Course ID

Response:

```
{
  "success": true,
  "data": {
    "id": "course-uuid",
    "name": "Pebble Beach Golf Links",
    "location": "Pebble Beach, CA",
    "address": "1700 17 Mile Dr",
```

```
    "phone": "+18316228723",
    "website": "https://pebblebeach.com",
    "rating": 4.8,
    "holes": 18,
    "par": 72,
    "yardage": 6828,
    "amenities": ["Pro Shop", "Restaurant", "Driving Range"],
    "isFavorite": false
  }
}
```

11.5 Get Favorite Courses

GET

`/api/courses/favorites`
Get user's favorite courses.
Access: Private

11.6 Add Favorite Course

POST

`/api/courses/favorites`
Add a course to favorites.
Access: Private
Request Body:

```
{
  "courseId": "course-uuid"
}
```

11.7 Remove Favorite Course

DELETE

`/api/courses/favorites/:id`
Remove a course from favorites.
Access: Private
URL Parameters:

- `id`: Course ID

12 Error Codes

Code	HTTP Status	Description
`UNAUTHORIZED`	401	Authentication required or token invalid
`FORBIDDEN`	403	User lacks permission for this action
`NOT_FOUND`	404	Resource not found
`VALIDATION_ERROR`	400	Request validation failed
`RATE_LIMITED`	429	Too many requests
`CONFLICT`	409	Resource conflict (e.g., duplicate)
`INTERNAL_ERROR`	500	Server error
`INVITE_EXPIRED`	400	Invite has expired

Code	HTTP Status	Description
TEE_TIME_FULL	400	Tee time is at capacity
ALREADY_RESPONDED	400	Already responded to this invite
BLOCKED_USER	403	User is blocked

13 Rate Limiting

The API implements rate limiting on certain endpoints to prevent abuse:

Endpoint	Limit
/auth/login	5 requests per minute
/auth/verify-otp	5 requests per minute
/auth/magic-link	3 requests per minute
/auth/refresh	10 requests per minute
General API	100 requests per minute

When rate limited, the API returns a 429 status code with a `Retry-After` header.

14 Webhooks & Notifications

The app uses push notifications for real-time updates. Notification types include:

- `friend_request` - New friend request received
- `friend_accepted` - Friend request accepted
- `tee_time_invite` - Invited to a tee time
- `invite_accepted` - Someone accepted your invite
- `invite_declined` - Someone declined your invite
- `invite_expired` - Your invite has expired
- `tee_time_reminder` - Upcoming tee time reminder
- `check_in_reminder` - Time to check in
- `tee_time_not_filled` - Tee time not filled, select from On-Deck
- `roster_invite_expired` - Roster invite expired, select from On-Deck